jigsaw-agile-community-rules

2位 Solution

Rank	2
Team	^^
Author	Yaoqing Jin
Topic ID	614279
Version	Japanese Translation

Source: https://www.kaggle.com/competitions/jigsaw-agile-community-rules/discussion/614279
Retrieved with Kaggle CLI on 2026-10-21.

Kaggle Discussionの主投稿と、技術的補足を含む選定済みQ&Aを日本語へ翻訳した版です。code、URL、model名、識別子、数値は原文を保持しています。

まず、この非常に興味深いコンペを企画し、現実的で価値の高い問題設定を公開してくださった Kaggle と Jigsaw チームに心から感謝します。このタスクは単なる「分類コンペ」ではありません。むしろ、極めてノイズが多く、ルールが変わりうるうえ、ラベルさえ常に信頼できるとは限らない、現実のコミュニティモデレーション環境にいきなり放り込まれるようなものです。

チームメイトにも感謝します。解法の設計、実装の細部、デバッグ、さらには notebook のバージョン管理のような地味な作業においても、チームメイトは非常に直接的で効果的な提案と健全性チェックをしてくれました。おかげで多くの時間の無駄を避けられました。特に終盤、学習と推論をどう並列化するか、そして "Notebook Threw Exception" や "Time Out" のような Kaggle のランダム性をどう事前に避けるかを議論していたとき、その協力は絶対に欠かせないものでした。

また、なぜ今になってこの write-up を投稿するのかも説明させてください。この2週間、本業が非常に忙しかったのです。日中のほとんどはプロジェクトの締め切りに追われ、コードやログを整理できるのは週末の夜だけでした。そのため、このレポートは今、日曜の夜まで遅れてしまいました。

手法の概要

jigsaw-llama3-1-8b-instruct-training-one-epoch が、基本的に私たちのアプローチの出発点でした。@mks2192 に深く感謝します。この notebook は非常に重要な事実を確認してくれました。unsloth は Kaggle 環境内でオンラインの LoRA ファインチューニングを行うのに非常に適しており、推論時にも非常に高いスループットを維持できる、ということです。私たちのその後のパイプラインは、本質的にはその上に積み重ねた反復的な進化でした。

最終提出は、複数モデルのアンサンブルによる LLM 判別器システムでした。核となる考え方は次のとおりです。

- このタスクを、対話形式で「LLM がルールとコメントを読み、Yes か No だけを答える」ように見える instruction tuning の問題として扱う。
- 教師信号の信頼性を高めるため、ノイズのあるアノテーションに対して明示的にラベルのクリーニングと重複除去を行う。
- subreddit のバイアスと汚染が極めて深刻だったため、モデルが subreddit フィールドに依存しないよう意図的に防ぐ。
- 異なるモデルファミリー・異なるサイズの複数のベース大規模言語モデルを使い、Kaggle 上で unsloth を使って LoRA でファインチューニングする。
- 推論時は、Yes と No の次トークンの logit の差を見て確率スコアを得る。
- すべてのモデルの予測に、単純だが頑健な加重融合を適用する。

1. ローカル検証戦略

コンペの初期に、オンライン学習では public leaderboard のスコアが非常に大きく変動し、予想外の問題も多く起きることが分かりました。たとえば、まったく同じ notebook が、通ることもあれば即座に Exception を投げることもありました。つまり、1日5回の提出だけに頼るのは非常に不安だということです。

学習データ上で、ラベルで層化し val_ratio = 0.1 とした StratifiedKFold の分割を作りました。言い換えると、二値ターゲット rule_violation で層化分割を行い、サンプルの約10%をローカル検証セットとしました。手法を変える

たびに、3つのランダムシードにわたるクロスバリデーションスコアの平均で評価しました。ローカル AUC を計算し、これをオフラインの指標として使いました。

この安定したローカルのクロスバリデーションスコアが、本質的にその後のすべての判断の拠り所でした。

2. ラベルのクリーニング

公式の学習データには実際上の問題がありました。同じ body（コメントのテキスト）と同じ rule（ルールのテキスト）の組が、場所によって異なる rule_violation の値でラベル付けされていることがあったのです。つまり、まったく同じコメントとルールの組が、違反とラベル付けされることもあれば、違反ではないとラベル付けされることもありました。これらの矛盾するサンプルで直接学習すると、LLM は「この問題には標準的な答えがない」と学習してしまい、教師信号が発散します。

クリーニングの手順は次のとおりです。

- 各行を1つのキーに圧縮する。2つのバリアントを試した。
  - "body || rule || subreddit || rule_violation"
  - そしてより積極的なバージョン: "body || rule || rule_violation"。つまり subreddit を無視する。
- 最終的に主に選んだのは subreddit を無視する方である。subreddit がなぜ問題かは、下の第3節で説明する。
- 同じ (body_trim, rule_trim) の組で矛盾するラベルが観測された場合、rule_violation が陽性である回数と総数を数える。
- 多数決を最終ラベルとする。
- 矛盾する行をこの多数派ラベルで埋める。
- その後、重複を除去する。

このクリーニングには2つの大きな利点があります。

- よりきれいな教師信号が得られる。LLM は教師ありファインチューニング中に、自己矛盾するように学習させられることがなくなる。
- サンプルあたりの品質が上がる。矛盾の解消と重複除去の後、データセットのサイズは実際には小さくなる。しかし、残った各サンプルはより信頼できる。低ランク適応の手法である LoRA にとっては、多くてもノイズの多いサンプルより、少なくてもきれいなサンプルの方が価値がある。特に、学習は1 epoch 分の予算しかないためである。

3. 意図的に subreddit 特徴を落とす

小さなプローブを行ったところ、学習データでは subreddit フィールドがしばしば強い社会的バイアスの信号を運んでいることが分かりました。同じルール・同じ行動でも、subreddit によって「違反」とも「違反ではない」とも判断されうるのです。基準、トーン、許容度はまったく異なり、ときには混沌としてさえいました。

判別的な LLM にとって、subreddit は非常に簡単にショートカット特徴、さらにはリーク特徴になります。

最終的に、次の方針を主なアプローチとすることに決めました。プロンプトから subreddit を取り除き、Rule: と Comment: だけを残しました。

プロンプトは基本的に次のようなものでした。

```
System:
"Decide if the Reddit comment violates the rule. Reply strictly with Yes or No."

User:
Rule: <rule text>
Comment: <comment text>

Assistant:
Yes / No
```

4. 複数モデルの並列ファインチューニングとアンサンブル

4.1 モデルファミリー

最終的に、異なるスケールと異なる LLM アーキテクチャをカバーする、4つの LoRA ファインチューニング版を学習して提出しました。

- bnb 4-bit 量子化でロードした phi-4-14b
- bnb 4-bit でロードした qwen3-14b
- bnb 4-bit でロードした qwen3-8b
- bnb 4-bit でロードした qwen25-7b-instruct

これらのモデルはすべて、unsloth.FastLanguageModel.from_pretrained(...) を使って 4-bit 量子化された重みでロードし、その後 LoraConfig で LoRA を適用しました。trl.SFTTrainer を使って、1 epoch の教師ありファインチューニングを行いました。

4.2 Kaggle 上の実務的なエンジニアリングの問題（GPU 並列化と再実行の不安定さ）

終盤には、4つのモデルすべてをエンドツーエンドで動かす必要がありました。これには学習、推論、submissionX.csv の生成、そして予測の融合が含まれます。これは Kaggle のリソース上限に非常に近いものでした。最終提出の総実行時間は11時間でした。

これを可能にするため、2つのエンジニアリング上の工夫をしました。

(1) デュアル GPU の並列スケジューリング all_train.sh というスクリプトを書き、モデルを2つのグループに分けました。それらを異なる CUDA_VISIBLE_DEVICES の下で起動し、2つのプロセスが同時に動けるようにしました。

このアプローチにより、総実行時間を Kaggle がまだ受け付ける範囲内に収められました。

(2) 「同じ notebook が動くこともあればクラッシュすることもある」という現実 非常にもどかしい Kaggle の再実行現象を目にしました。まったく同じ notebook のバージョンが、完走してスコアを出すこともあれば、即座に Notebook Threw Exception や Time Out を投げることもあったのです。私たちの仮説は、Kaggle が提出ごとに常にまったく同じ基盤のハードウェアリソースを割り当てているわけではない、というものです。

緩和策は次のとおりです。

- 学習の epoch 数を1に固定する。
- 「十分大きいが OOM を起こさない」バッチサイズ、たとえば per_device_train_batch_size = 4 または 8 を選び、gradient_accumulation_steps で実効的なグローバルバッチサイズを制御する。

- 推論時は、極端に大きなバッチサイズに攻めるのではなく、安全な値である bs=4 または bs=per_device_train_batch_size*2 を使う。

言い換えると、スクリプトを意図的に調整して、100% まで攻めてメモリ不足のエラーを起こすリスクを取るのではなく、利用可能なリソースの約80〜90% を使うようにしました。これにより、変動する環境の下でも生き残る可能性が高くなりました。

改めて、Kaggle、Jigsaw チーム、そしてベースラインについて議論し共有してくれたすべての参加者に感謝します。コミュニティモデレーションとコンテンツポリシーの判断というこの領域は、現実的で難しく、少し混沌としてさえいます。だからこそ、取り組み続ける価値があるのです。